

Documentation

Scène 3D en langage C

Réalisée par : Meradji Anaïs

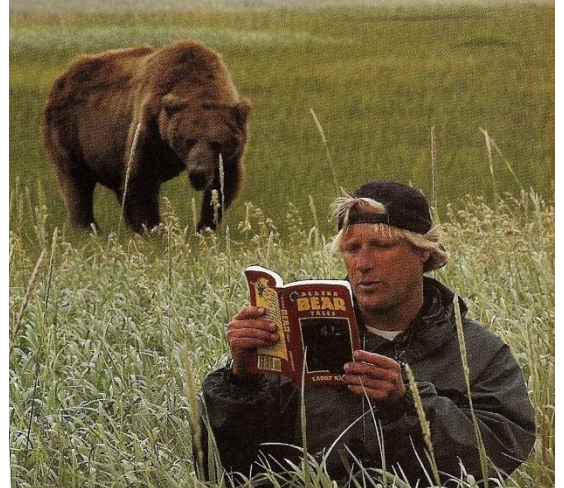
1. Contexte : histoire de la scène (la source d'inspiration)

Timothy Treadwell était un écologiste américain passionné par les ours. Il a passé plus de 13 étés dans le parc national de Katmai en Alaska, où il étudiait et vivait parmi les ours bruns, sans aucune protection.

Timothy Treadwell a développé une fascination pour les ours bruns dès son plus jeune âge. Il les considérait comme des créatures majestueuses et voulait consacrer sa vie à les protéger.

Pour se rapprocher des ours bruns, Timothy Treadwell adoptait une méthode risquée et controversée. Il s'approchait des animaux de très près, parfois à seulement quelques mètres, sans se soucier des avertissements des experts sur le danger que cela représentait.

Il pensait avoir développé une relation unique avec les ours, affirmant qu'ils le considéraient comme un membre de leur famille.



Par ailleurs, il appréciait également beaucoup les renards.



2003 : pour le 13 -ème été consécutive, Timothy s'apprêtait à passer son été au parc national de Katmai (Alaska), cette fois ci accompagné de sa petite amie Amie Huguenard.

5 octobre 2003 :

Willy Fulton, pilote censé récupérer Timothy et Amy du parc national de Katmai, fit une tragique découverte en atterrissant "Un grand ours était assis au sommet d'un tas de restes humains, s'est-il souvenu. Il était en train de manger une cage thoracique humaine."

Les tentes de Timothy et d'Amie ont été retrouvées déchirées. Près de leur couchage, la police a retrouvé des restes humains ainsi que la tête mutilée de Timothy Treadwell.

Il fut établi que Timothy Treadwell et Amy fut attaqués par un grizzli, et dévorés vivant.

Et ainsi fut la fin de l'aventurier **Timothy Treadwell**.
Un passionné.

Il serait réducteur de se souvenir de lui comme l'homme dévoré par un ours, car c'est aussi l'homme qui a survécu 13 étés parmi les ours.

L'homme qui s'est isolé de l'humanité pour vivre une vie que l'on peut seulement imaginer.

L'homme qui a vraiment vécu.

The grizzly man.



2. Création de la scène parc Katmai

- Environnement de travail :

Codeblocks.

-[Bibliothèque freeglut](#) : bibliothèque qui gère la fenêtre, le clavier, la souris et la boucle d'affichage pour les programmes OpenGL.

-[Bibliothèque glew](#) : bibliothèque qui charge automatiquement les extensions OpenGL modernes et permet d'utiliser des fonctionnalités absentes du pipeline fixe, comme les shaders et les VBO.

Shaders : les shaders en OpenGL sont de petits programmes exécutés directement sur la carte graphique pour contrôler comment les vertices et les pixels sont transformés et affichés. Ils remplacent l'ancien pipeline fixe et permettent d'avoir un rendu personnalisé, comme les lumières, couleurs, effets, textures, etc.

```
#include <GL/glew.h>
#include <GL/glut.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdbool.h>
#include <time.h>

#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"

#ifndef GL_CLAMP_TO_EDGE
#define GL_CLAMP_TO_EDGE 0x812F
#endif

#define MAX_PARTICLES 300
```

Ci-dessus les Include dont nous aurons besoin.

Comme mentionné précédemment, **glew** et **glut**. Ensuite **stdio.h** pour les entrées/sorties standard, **stdlib.h** pour les allocations mémoire, **string.h** pour la manipulation des chaînes, **math.h** pour les fonctions mathématiques (seront utiles pour l'animation de certains éléments de la scène), **stdbool.h** pour déclarer des booléens, et enfin **time.h** pour la gestion du temps.

Par ailleurs, nous allons inclure **stb_image.h** qui nous permettra de faire charger des images afin d'appliquer des textures sur des objets de la scène.

Je définis également une variable `max_particles`, fixée à 300 (ça sera utilisé pour ajouter de la neige à la scène, car c'est en Alaska après tout.. Katmai)

- Variables :

```
// STRUCTURES

typedef struct {
    float x, y, z;
    float speedY;
    float speedX;
    bool active;
} Particle;

typedef struct {
    char name[128];
    float ambient[3];
    float diffuse[3];
    float specular[3];
    float shininess;
    char textureFile[256];
    GLuint textureID;
} Material;

typedef struct {
    float* vertices;
    float* normals;
    float* texcoords;
    int* faces;
    int* materialIndices;
    int vertexCount;
    int normalCount;
    int texcoordCount;
    int faceCount;
    Material* materials;
    int materialCount;
} OBJModel;
```

On commence par définir les structures dont nous aurons besoin.

Particle : Représente une particule de neige

- `x, y, z` : position 3D
- `speedY, speedX` : vitesse de chute
- `active` : particule visible ou non

Material : Propriétés d'un matériau

- `ambient[]` : couleur ambiante (lumière indirecte)
- `diffuse[]` : couleur diffuse (lumière directe)
- `specular[]` : couleur spéculaire (reflets brillants)
- `shininess` : intensité des reflets
- `textureFile[]` : chemin vers la texture
- `textureID` : identifiant OpenGL de la texture

OBJModel : Modèle 3D chargé depuis fichier .obj

- `vertices[]` : coordonnées des sommets (x,y,z)
- `normals[]` : vecteurs normaux pour l'éclairage
- `texcoords[]` : coordonnées de texture (u,v)
- `faces[]` : indices définissant les triangles
- `materialIndices[]` : matériau de chaque face
- Compteurs : `vertexCount`, `normalCount`, etc.

Puis nous avons des variables a initialiser :

```
GLuint texBear, texFox, texGrass, texMountain, texSky, texTree, texTronc, texRock, texRiver, texSalmon, texEagle;

Particle particles[MAX_PARTICLES];

OBJModel bear1Model = {0};
OBJModel bear2Model = {0};
OBJModel foxModel = {0};
OBJModel timothyModel = {0};
OBJModel salmonModel = {0};
OBJModel eagleModel = {0};
OBJModel riverModel = {0};

float timeElapsed = 0.0f;
float waterOffset = 0.0f;
float dayNightCycle = 0.0f;

float foxX = -5.0f;
float foxZ = -4.0f;
float foxAngle = 90.0f;
float foxSpeed = 0.01f;
int foxDirection = 1;
float foxLegAngle = 0.0f;
bool foxAutoMove = true;
bool foxMovedManually = false;
bool foxMovedVertically = false;
```

Les textures, les objets, les positions et angles..

```
float cameraDistance = 10.0f;
float cameraHeight = 2.5f;
float cameraAngleX = 0.0f;
float cameraAngleY = 0.0f;
float targetCameraDistance = 10.0f;
float zoomSpeed = 1.0f;
int lastMouseX = 0;
int lastMouseY = 0;
bool mouseRotating = false;

bool light1On = true;
bool light2On = false;
bool autoDayNight = true;

const float INITIAL_CAMERA_DISTANCE = 10.0f;
const float INITIAL_CAMERA_HEIGHT = 2.5f;
const float INITIAL_CAMERA_ANGLE_X = 0.0f;
const float INITIAL_CAMERA_ANGLE_Y = 0.0f;

// VARIABLES SHADERS

GLuint shaderProgram = 0;
int illuminationMode = 3;
int lightType = 0;
bool useShadersForScene = true;
```

Variables de camera (distance, angle), booléens pour les lumières..

Et pour finir des variables shaders :

- GLuint shaderProgram = 0; : identifiant OpenGL du programme de shaders (0 = aucun programme chargé).
- int illuminationMode = 3; : mode d'éclairage choisi (valeur entière servant à basculer entre plusieurs modèles d'illumination).
- int lightType = 0; : type de source lumineuse (entier indiquant point, directionnelle, spot).
- bool useShadersForScene = true; : indique si la scène doit être rendue avec des shaders (true) ou le pipeline fixe (false).

La dernière variable booléenne pour les shaders permettra de comparer la scene rendue avec shaders VS sans shaders.

Passons aux fonctions :

```
// FONCTIONS SHADERS

char* loadShaderSource(const char* filename) {
    FILE* file = fopen(filename, "rb");
    if (!file) {
        printf("ERREUR: Impossible d'ouvrir le shader %s\n", filename);
        return NULL;
    }

    fseek(file, 0, SEEK_END);
    long length = ftell(file);
    fseek(file, 0, SEEK_SET);

    char* source = (char*)malloc(length + 1);
    if (!source) {
        fclose(file);
        return NULL;
    }

    fread(source, 1, length, file);
    source[length] = '\0';
    fclose(file);

    return source;
}
```

Cette fonction (**loadShaderSource**) lit un fichier contenant du code shader (shader.frag, qu'on montrera plus tard) et le charge entièrement en mémoire. Elle ouvre le fichier, récupère sa taille, lit son contenu et renvoie le texte sous forme de chaîne C. Si un problème survient, elle renvoie NULL.

```
GLuint compileShader(GLenum type, const char* source) {
    GLuint shader = glCreateShader(type);
    glShaderSource(shader, 1, &source, NULL);
    glCompileShader(shader);

    GLint success = 0;
    glGetShaderiv(shader, GL_COMPILE_STATUS, &success);

    if (!success) {
        char infoLog[512];
        glGetShaderInfoLog(shader, 512, NULL, infoLog);
        printf("ERREUR COMPILATION SHADER:\n%s\n", infoLog);
        glDeleteShader(shader);
        return 0;
    }

    return shader;
}
```

Cette fonction compile un shader (vertex ou fragment) à partir de son code source. Elle crée l'objet shader, lui attribue le texte, lance la compilation et vérifie si elle a réussi. En cas d'erreur, elle affiche le message de compilation et renvoie 0.

```
GLuint createShaderProgram(const char* vertPath, const char* fragPath) {
    char* vertSource = loadShaderSource(vertPath);
    char* fragSource = loadShaderSource(fragPath);

    if (!vertSource || !fragSource) {
        printf("ERREUR: Impossible de charger les shaders\n");
        if (vertSource) free(vertSource);
        if (fragSource) free(fragSource);
        return 0;
    }

    GLuint vertShader = compileShader(GL_VERTEX_SHADER, vertSource);
    GLuint fragShader = compileShader(GL_FRAGMENT_SHADER, fragSource);

    free(vertSource);
    free(fragSource);

    if (!vertShader || !fragShader) {
        if (vertShader) glDeleteShader(vertShader);
        if (fragShader) glDeleteShader(fragShader);
        return 0;
    }

    GLuint program = glCreateProgram();
    glAttachShader(program, vertShader);

    glAttachShader(program, fragShader);
    glLinkProgram(program);

    GLint success = 0;
    glGetProgramiv(program, GL_LINK_STATUS, &success);

    if (!success) {
        char infoLog[512];
        glGetProgramInfoLog(program, 512, NULL, infoLog);
        printf("ERREUR LINKAGE PROGRAMME:\n%s\n", infoLog);
        glDeleteProgram(program);
        program = 0;
    }

    glDeleteShader(vertShader);
    glDeleteShader(fragShader);

    return program;
}
```

Cette fonction assemble un vertex shader et un fragment shader pour créer un programme complet utilisable par OpenGL. Elle charge les fichiers, compile les deux shaders, les attache à un programme et effectue l'étape de linkage. Si tout réussit, elle renvoie un identifiant de programme prêt à être utilisé.

Résumé du pipeline shader :

- Le programme charge le code des shaders depuis des fichiers (vertex et fragment).
- Chaque shader est ensuite compilé séparément par OpenGL.
- Si la compilation fonctionne, les deux shaders sont attachés à un programme GPU.
- Le programme shader est lié (linkage) pour former un pipeline complet utilisable par OpenGL.
- Une fois le programme lié, les objets shader individuels sont supprimés.
- Le programme final peut alors être activé pour remplacer le pipeline fixe lors du rendu.

On met shader.frag et shader.vert là où s'exécute notre scène.

> Ce PC > Téléchargements > freeglut-MinGW-3.0.0-1.mp > freeglut > bin				
☐	Nom	Modifié le	Type	Taille
	x64	08/10/2025 16:24	Dossier de fichiers	
	freetglut.dll	08/10/2025 16:24	Extension de l'app...	222 Ko
	shader.frag	12/12/2025 12:30	Fichier FRAG	3 Ko
	shader.vert	12/12/2025 12:25	Fichier VERT	1 Ko

Shader.frag :

```
#version 120

// Inputs depuis vertex shader
varying vec3 vNormal;
varying vec3 vPosition;
varying vec2 vTexCoord;
varying vec4 vColor; // recevoir la couleur du vertex

// Uniforms
uniform int uMode; // 1=Lambertien, 2=Speculaire, 3
=Glossy
uniform int uLightType; // 0=Directionnelle, 1=Pointuelle,
2=Spot
uniform int uUseTexture; // 1=utiliser texture, 0=utiliser
couleur

uniform vec3 uLightDir;
uniform vec3 uLightPos;
uniform vec3 uSpotDir;
uniform float uSpotInnerCut;
uniform float uSpotOuterCut;
uniform vec3 uViewPos;

uniform sampler2D texture0;

void main() {
    vec3 N = normalize(vNormal);
    vec3 L;
    float attenuation = 1.0;

    // CALCUL DIRECTION LUMIÈRE

    if (uLightType == 0) {
        // Directionnelle
        L = normalize(uLightDir);
    } else if (uLightType == 1) {
        // Pointuelle
        vec3 lightVec = uLightPos - vPosition;
        float distance = length(lightVec);
        L = normalize(lightVec);
        attenuation = 1.0 / (1.0 + 0.09 * distance + 0.032 *
distance * distance);
    } else if (uLightType == 2) {
        // Spot
        vec3 spotDir = normalize(uSpotDir);
        float cos2theta = 1 - uSpotInnerCut;
        float cos2theta0 = 1 - uSpotOuterCut;
        float randCos2theta = sqrt(cos2theta0 * cos2theta);
        float cosTheta = sqrt(1 - randCos2theta);
        L = normalize(uSpotDir * cosTheta +
        vec3(0, 0, 1) * sinTheta);
    }

    vec3 R = L * vNormal * 2 - L;
    vec3 V = uViewPos - vPosition;
    float ndotL = max(0, dot(N, L));
    float ndotR = max(0, dot(N, R));
    float ndotV = max(0, dot(N, V));
    float fR = uMode == 2 ? ndotL : uMode == 3 ? ndotV : 1;
    vec3 c = fR * vColor + (1 - fR) * texture0(vTexCoord);
    gl_FragColor = c;
}
```

```

    } else {
        // Spot
        vec3 lightVec = uLightPos - vPosition;
        float distance = length(lightVec);
        L = normalize(lightVec);

        attenuation = 1.0 / (1.0 + 0.09 * distance + 0.032 *
distance * distance);

        vec3 spotDirection = normalize(uSpotDir);
        float theta = dot(L, -spotDirection);
        float epsilon = uSpotInnerCut - uSpotOuterCut;
        float intensity = clamp((theta - uSpotOuterCut) /
epsilon, 0.0, 1.0);
        attenuation *= intensity;
    }

    // COMPOSANTE AMBIANTE
    vec3 ambient = vec3(0.3, 0.3, 0.35);

    // COMPOSANTE DIFFUSE (Lambertien)

    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diff * vec3(1.0, 1.0, 1.0);

    // COMPOSANTE SPÉCULAIRE

    vec3 specular = vec3(0.0);

    if (uMode == 2 || uMode == 3) {
        vec3 V = normalize(uViewPos - vPosition);
        vec3 R = reflect(-L, N);

        float shininess = (uMode == 2) ? 128.0 : 32.0;
        float spec = pow(max(dot(R, V), 0.0), shininess);
        specular = spec * vec3(0.5, 0.5, 0.5);
    }

    // COMBINAISON FINALE
    vec3 lighting = ambient + (diffuse + specular) *
attenuation;

    // Choisir entre texture et couleur du matériau

    vec4 texColor = texture2D(texture0, vTexCoord);
    vec3 baseColor;

    // Si la texture a de l'alpha > 0.1, on l'utilise, sinon
couleur du vertex

    if (uUseTexture == 1 && texColor.a > 0.1) {
        baseColor = texColor.rgb;
    } else {
        baseColor = vColor.rgb;
    }

    vec3 result = lighting * baseColor;

    gl_FragColor = vec4(result, 1.0);
}

```

Ce fragment shader calcule la couleur finale de chaque pixel selon le type de lumière (directionnelle, ponctuelle ou spot) et le mode d'illumination choisi (diffus, spéculaire ou

glossy). Il combine plusieurs composantes lumineuses : ambiante, diffuse et spéculaire, tout en tenant compte de l'atténuation selon la distance ou le cône du spot. Enfin, il applique cette lumière soit à la texture du modèle, soit à sa couleur, pour produire la couleur finale envoyée à l'écran.

Shader.vert :

```
#version 120

varying vec3 vNormal;
varying vec3 vPosition;
varying vec2 vTexCoord;
varying vec4 vColor; // transmettre la couleur au fragment
                        shader

void main() {
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    vPosition = (gl_ModelViewMatrix * gl_Vertex).xyz;
    vNormal = normalize(gl_NormalMatrix * gl_Normal);
    vTexCoord = gl_MultiTexCoord0.xy;
    vColor = gl_Color; // transmettre la couleur du vertex
}
```

Le vertex shader transforme chaque sommet du modèle en coordonnées écran et calcule sa position et sa normale dans l'espace caméra. Il transmet ensuite au fragment shader les informations nécessaires : position, normale, coordonnées de texture et couleur du vertex. Cela permet au fragment shader de calculer l'éclairage et la couleur finale de chaque pixel.

Maintenant nous allons définir 3 fonctions : une pour charger les textures sur la scène, une pour charger les .mtl (mapping entre les textures/couleurs et notre .obj) et une dernière fonction pour charger un objet 3d (.obj)

```
// CHARGEMENT TEXTURES
GLuint loadTexture(const char* filename) {
    GLuint textureID = 0;
    int width, height, nrChannels;

    FILE* testFile = fopen(filename, "rb");
    if (!testFile) return 0;
    fclose(testFile);

    stbi_set_flip_vertically_on_load(true);
    unsigned char* data = stbi_load(filename, &width, &height, &nrChannels, 0);
    if (!data) return 0;

    glGenTextures(1, &textureID);
    glBindTexture(GL_TEXTURE_2D, textureID);
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    GLenum format = (nrChannels == 4) ? GL_RGBA : GL_RGB;
    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format, GL_UNSIGNED_BYTE, data);

    stbi_image_free(data);
    return textureID;
}
```

Cette fonction charge une image depuis un fichier et crée une texture OpenGL prête à l'usage. Elle vérifie l'existence du fichier, lit les pixels avec la bibliothèque STB, crée une texture GPU, définit ses paramètres (répétition, filtrage) et la charge en mémoire GPU. L'ID retourné permet ensuite de l'utiliser dans le shader pour texturer un objet.

```
// CHARGEMENT MTL

void loadMTL(const char* path, OBJModel* model) {
    FILE* file = fopen(path, "r");
    if (!file) return;

    char line[512];
    int maxMaterials = 50;
    model->materials = (Material*)malloc(sizeof(Material) * maxMaterials);
    model->materialCount = 0;

    Material* currentMat = NULL;
    char basePath[256];
    strcpy(basePath, path);
    char* lastSlash = strrchr(basePath, '\\');
    if (!lastSlash) lastSlash = strrchr(basePath, '/');
    if (lastSlash) *(lastSlash + 1) = '\0';
    else basePath[0] = '\0';

    while (fgets(line, sizeof(line), file)) {
        if (strncmp(line, "newmtl ", 7) == 0) {
            currentMat = &model->materials[model->materialCount++];
            sscanf(line, "newmtl %127s", currentMat->name);
            currentMat->ambient[0] = currentMat->ambient[1] = currentMat->ambient[2] = 0.2f;
            currentMat->diffuse[0] = currentMat->diffuse[1] = currentMat->diffuse[2] = 0.8f;
            currentMat->specular[0] = currentMat->specular[1] = currentMat->specular[2] = 0.0f;
            currentMat->shininess = 0.0f;
            currentMat->textureFile[0] = '\0';
            currentMat->textureID = 0;
        }
        else if (currentMat && strncmp(line, "Ka ", 3) == 0) {
            sscanf(line, "Ka %f %f %f", &currentMat->ambient[0], &currentMat->ambient[1], &currentMat->ambient[2]);
        }
        else if (currentMat && strncmp(line, "Kd ", 3) == 0) {
            sscanf(line, "Kd %f %f %f", &currentMat->diffuse[0], &currentMat->diffuse[1], &currentMat->diffuse[2]);
        }
        else if (currentMat && strncmp(line, "Ks ", 3) == 0) {
            sscanf(line, "Ks %f %f %f", &currentMat->specular[0], &currentMat->specular[1], &currentMat->specular[2]);
        }
        else if (currentMat && strncmp(line, "Ns ", 3) == 0) {
            sscanf(line, "Ns %f", &currentMat->shininess);
        }
        else if (currentMat && strncmp(line, "map_Kd ", 7) == 0) {
            char texName[256];
            sscanf(line, "map_Kd %255s", texName);
            char fullPath[512];
            sprintf(fullPath, "%s%s", basePath, texName);
            strcpy(currentMat->textureFile, fullPath);
            currentMat->textureID = loadTexture(fullPath);
        }
    }

    fclose(file);
}
```

Cette fonction lit un fichier MTL pour charger les matériaux d'un modèle OBJ. Elle parcourt chaque ligne, crée un nouveau matériau pour chaque newmtl, récupère ses couleurs ambiante, diffuse et spéculaire, sa brillance et le fichier de texture associé. Les textures sont ensuite chargées en mémoire GPU pour être utilisées lors du rendu.

```
// CHARGEMENT OBJ
```

```
void loadOBJ(const char* path, OBJModel* model, bool loadMaterials) {
    FILE* file = fopen(path, "r");
    if (!file) return;

    char line[512];
    int maxVertices = 200000;
    int maxNormals = 200000;
    int maxTexcoords = 200000;
    int maxFaces = 300000;

    model->vertices = (float*)malloc(sizeof(float) * 3 * maxVertices);
    model->normals = (float*)malloc(sizeof(float) * 3 * maxNormals);
    model->texcoords = (float*)malloc(sizeof(float) * 2 * maxTexcoords);
    model->faces = (int*)malloc(sizeof(int) * 9 * maxFaces);
    model->materialIndices = (int*)malloc(sizeof(int) * maxFaces);

    model->vertexCount = 0;
    model->normalCount = 0;
    model->texcoordCount = 0;
    model->faceCount = 0;
    model->materialCount = 0;

    int currentMaterialIndex = -1;
    char mtlFile[256] = "";

    while (fgets(line, sizeof(line), file)) {
        if (strncmp(line, "mtllib ", 7) == 0 && loadMaterials && mtlFile[0] == '\0') {
            char* mtlStart = line + 7;
            while (*mtlStart == ' ' || *mtlStart == '\t') mtlStart++;

            mtlFile[i] = mtlStart[i];
            i++;
        }
        mtlFile[i] = '\0';

        char basePath[256];
        strcpy(basePath, path);
        char* lastSlash = strrchr(basePath, '\\');
        if (!lastSlash) lastSlash = strrchr(basePath, '/');
        if (lastSlash) *(lastSlash + 1) = '\0';
        else basePath[0] = '\0';
        char fullMtlPath[512];
        sprintf(fullMtlPath, "%s%s", basePath, mtlFile);
        loadMTL(fullMtlPath, model);
    }
    else if (strncmp(line, "usemtl ", 7) == 0 && loadMaterials) {
        char matName[128];
        sscanf(line, "usemtl %127s", matName);
        for (int i = 0; i < model->materialCount; i++) {
            if (strcmp(model->materials[i].name, matName) == 0) {
                currentMaterialIndex = i;
                break;
            }
        }
    }
    else if (strncmp(line, "v ", 2) == 0) {
        float x, y, z;
        if (sscanf(line, "v %f %f %f", &x, &y, &z) == 3) {
            model->vertices[model->vertexCount * 3 + 0] = x;
            model->vertices[model->vertexCount * 3 + 1] = y;
            model->vertices[model->vertexCount * 3 + 2] = z;
        }
    }
}
```

```

    else if (strcmp(line, "vt ", 3) == 0) {
        float u, v;
        if (sscanf(line, "vt %f %f", &u, &v) == 2) {
            model->texcoords[model->texcoordCount * 2 + 0] = u;
            model->texcoords[model->texcoordCount * 2 + 1] = v;
            model->texcoordCount++;
        }
    }
    else if (strcmp(line, "vn ", 3) == 0) {
        float nx, ny, nz;
        if (sscanf(line, "vn %f %f %f", &nx, &ny, &nz) == 3) {
            model->normals[model->normalCount * 3 + 0] = nx;
            model->normals[model->normalCount * 3 + 1] = ny;
            model->normals[model->normalCount * 3 + 2] = nz;
            model->normalCount++;
        }
    }
    else if (strcmp(line, "f ", 2) == 0) {
        int v[3] = {0,0,0}, vt[3] = {0,0,0}, n[3] = {0,0,0};

        int parsed = sscanf(line, "f %d/%d/%d %d/%d/%d %d/%d/%d",
            &v[0], &vt[0], &n[0], &v[1], &vt[1], &n[1], &v[2], &vt[2], &n[2]);

        if (parsed != 9) {
            parsed = sscanf(line, "f %d//%d %d//%d %d//%d",
                &v[0], &n[0], &v[1], &n[1], &v[2], &n[2]);
            vt[0] = vt[1] = vt[2] = 0;
        }

        if (parsed == 9 || parsed == 6) {
            bool valid = true;

            for (int i = 0; i < 3; i++) {
                if (v[i] < 1 || v[i] > model->vertexCount) valid = false;
                if (n[i] < 1 || n[i] > model->normalCount) valid = false;
                if (vt[i] > 0 && vt[i] > model->texcoordCount) valid = false;
            }
            if (!valid) continue;

            for (int i = 0; i < 3; i++) {
                model->faces[model->faceCount * 9 + i*3 + 0] = v[i] - 1;
                model->faces[model->faceCount * 9 + i*3 + 1] = vt[i] - 1;
                model->faces[model->faceCount * 9 + i*3 + 2] = n[i] - 1;
            }
            model->materialIndices[model->faceCount] = currentMaterialIndex;
            model->faceCount++;
        }
    }
}

fclose(file);

```

Cette fonction lit un fichier OBJ pour charger la géométrie d'un modèle 3D : sommets, normales, coordonnées de texture et faces.

Rappel :

- **Sommet (Vertex)** : Un point dans l'espace 3D qui définit la géométrie d'un modèle.
- **Coordonnées de texture (Texcoords / UV)** : Points 2D qui indiquent comment appliquer une image (texture) sur la surface du modèle.
- **Normale (Normal)** : Vecteur perpendiculaire à une surface, utilisé pour calculer l'éclairage et les ombres.
- **Face (Face / Triangle)** : Ensemble de 3 sommets (ou plus) formant une surface du modèle 3D.

Elle gère également les matériaux via le fichier MTL lié, et associe chaque face à son matériau correspondant. Les indices sont convertis pour OpenGL et stockés dans des tableaux pour le rendu.

Système de particules pour la neige :

```
// SYSTÈME DE PARTICULES

void initParticles() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PARTICLES; i++) {
        particles[i].x = (float)(rand() % 100 - 50);
        particles[i].y = (float)(rand() % 30 + 5);
        particles[i].z = (float)(rand() % 40 - 20);
        particles[i].speedY = -0.02f - (float)(rand() % 10) / 1000.0f;
        particles[i].speedX = (float)(rand() % 10 - 5) / 1000.0f;
        particles[i].active = true;
    }
}

void updateParticles() {
    for (int i = 0; i < MAX_PARTICLES; i++) {
        if (particles[i].active) {
            particles[i].y += particles[i].speedY;
            particles[i].x += particles[i].speedX;

            if (particles[i].y < 0.0f) {
                particles[i].y = 25.0f + (float)(rand() % 10);
                particles[i].x = (float)(rand() % 100 - 50);
                particles[i].z = (float)(rand() % 40 - 20);
            }
        }
    }
}
```

Le système de particules crée un ensemble de petites entités animées simulant des effets comme la pluie ou la fumée.

Chaque particule a une position et une vitesse, est mise à jour à chaque frame, et lorsqu'elle tombe sous une certaine limite, elle est remplacée en haut pour créer un effet continu.

- Dessins :

Nous avons :

drawParticles()

Affiche toutes les particules actives comme des points transparents pour simuler des effets comme la pluie ou la fumée.

Chaque particule est positionnée selon ses coordonnées et se fond dans la scène grâce au mélange alpha.

drawGround()

Dessine le sol texturé en herbe en utilisant une grille de quads.

Chaque quad reçoit une portion de texture pour créer un sol continu et réaliste.

drawSky()

Dessine le ciel texturé autour de la caméra avec une couleur qui varie selon le cycle jour/nuit.

Utilise une sphère texturée et ignore la profondeur pour donner l'illusion d'un ciel lointain.

drawTree(float x, float z, float height)

Dessine un arbre à une position donnée, avec un tronc cylindrique et des feuilles coniques empilées.

Les textures sont appliquées sur le tronc et les feuilles pour plus de réalisme.

drawRock(float x, float y, float z, float size)

Affiche un rocher texturé à une position et taille données.

Utilise une sphère transformée (scalée et tournée) pour simuler la forme irrégulière du rocher.

drawMountainRange(float startX, float z, float baseWidth, float height)

Dessine une chaîne de montagnes segmentée avec des sommets irréguliers.

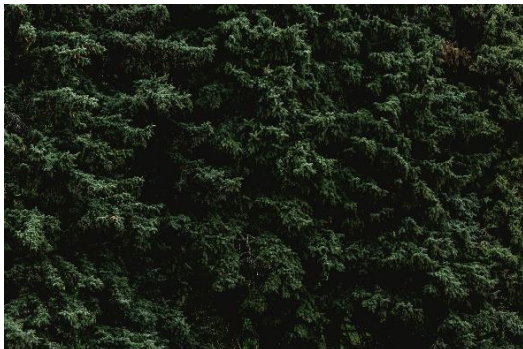
La hauteur de chaque pic est modulée pour créer un relief naturel et texturé.

drawAllMountains()

Compose plusieurs chaînes de montagnes pour créer un paysage complet.

Applique des rotations et translations pour disposer correctement les différentes chaînes dans la scène.

Chacun de ces objets est texturé de façon réaliste.



- Insertion d'objet 3D :

Nous avons plusieurs objets 3D sur notre scène :

Deux ours, deux renards, plusieurs poissons, une rivière, et un être humain (représentant Timothy Treadwell)

-Utilisation de **Sketchfab** pour téléchargement de modèles 3D classiques.

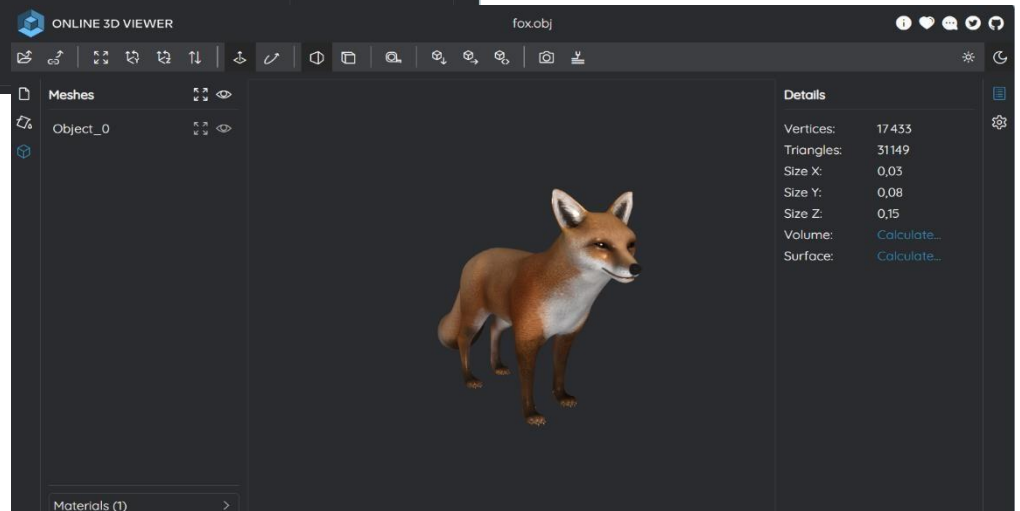
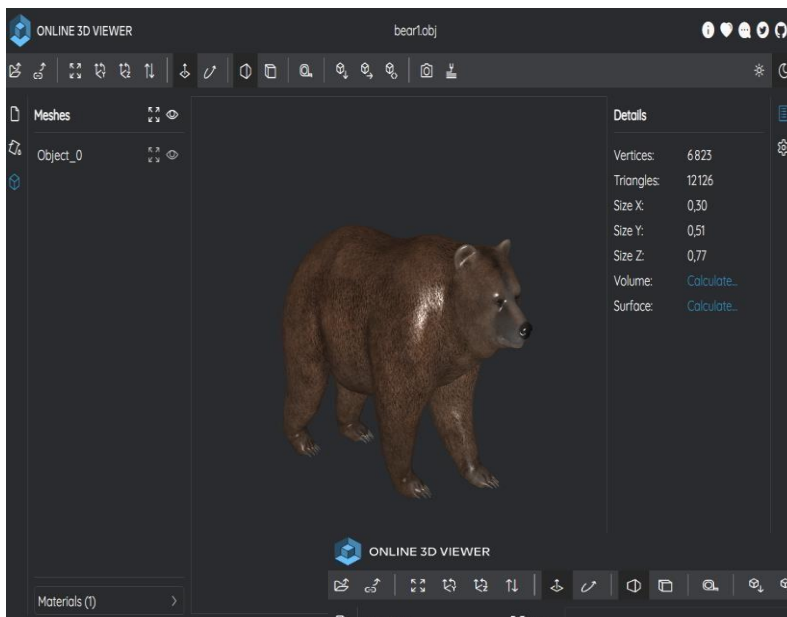
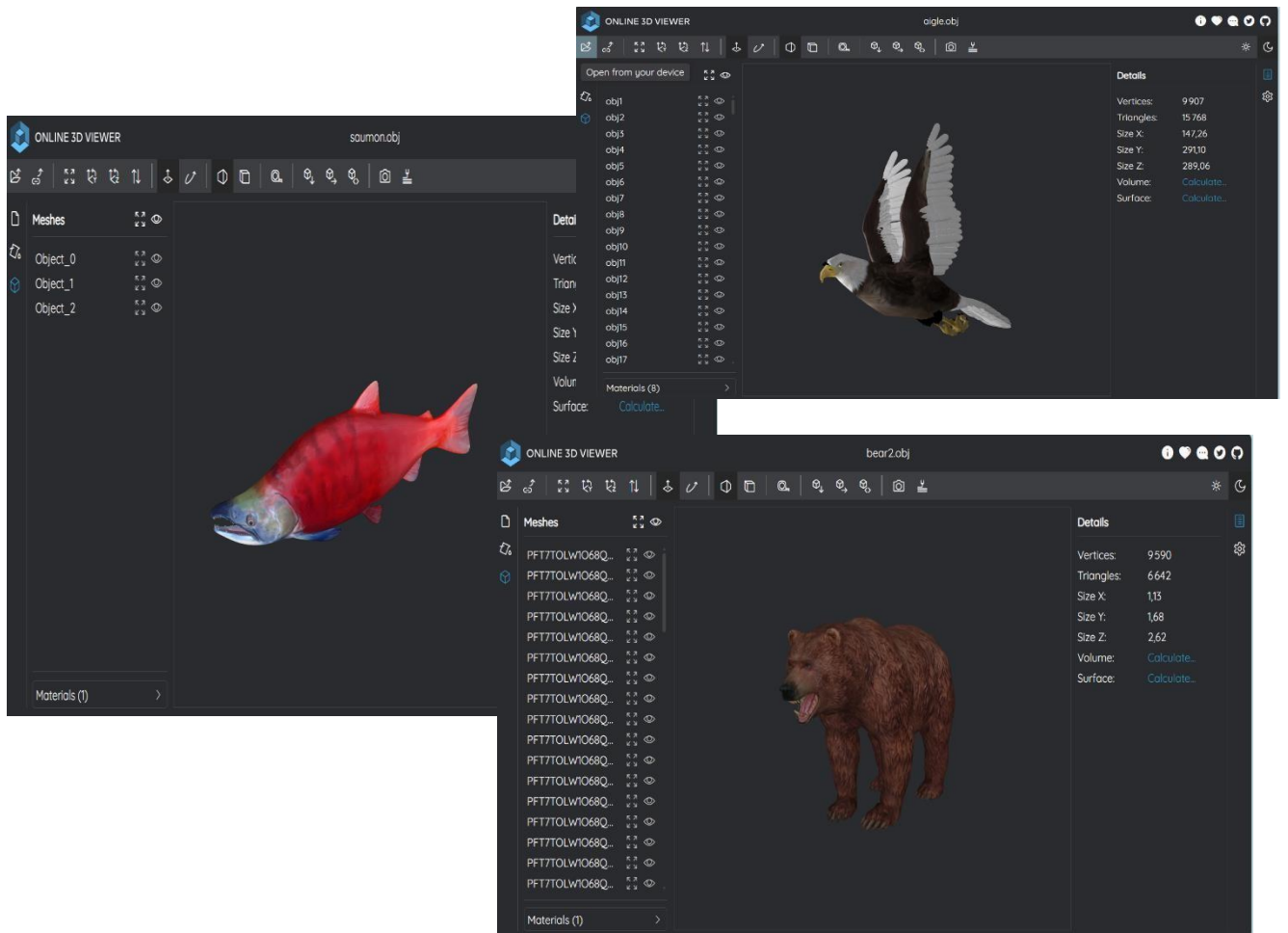
-Utilisation de **3d viewer** pour vérification de la bonne application des textures via .mtl .

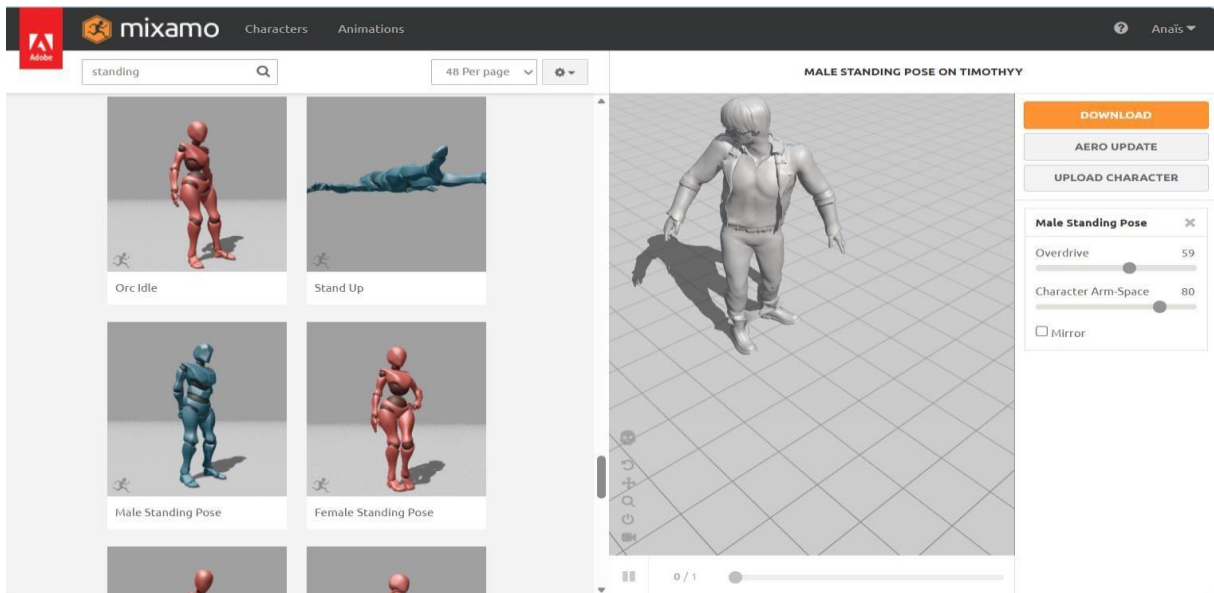
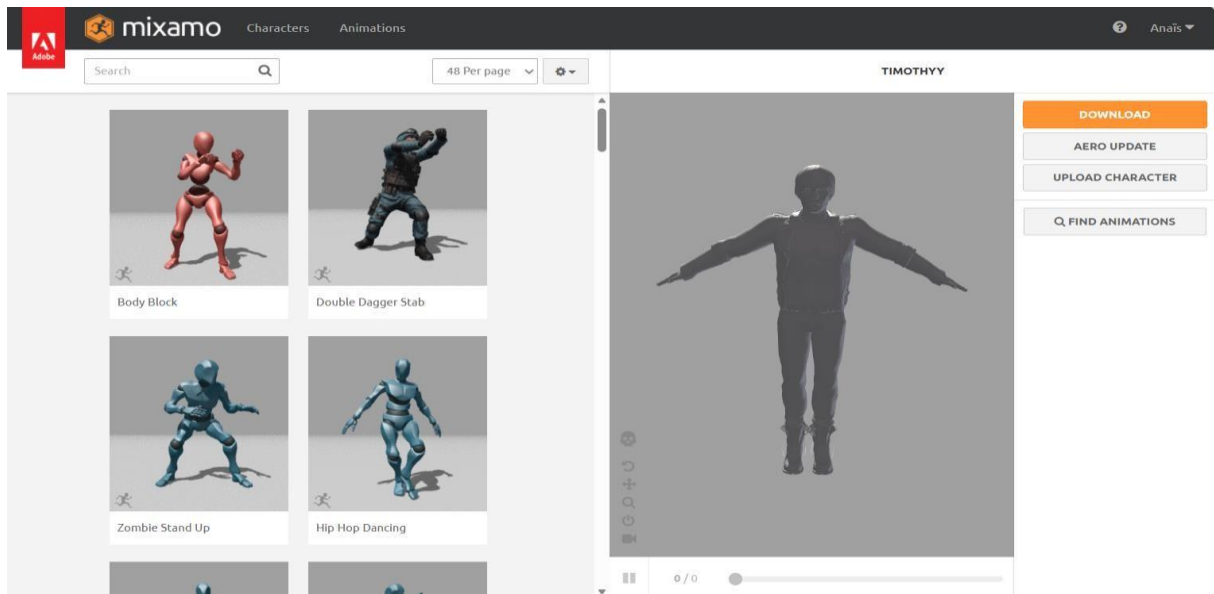
-Utilisation de **Blender** pour le personnage humain Timothy Treadwell.

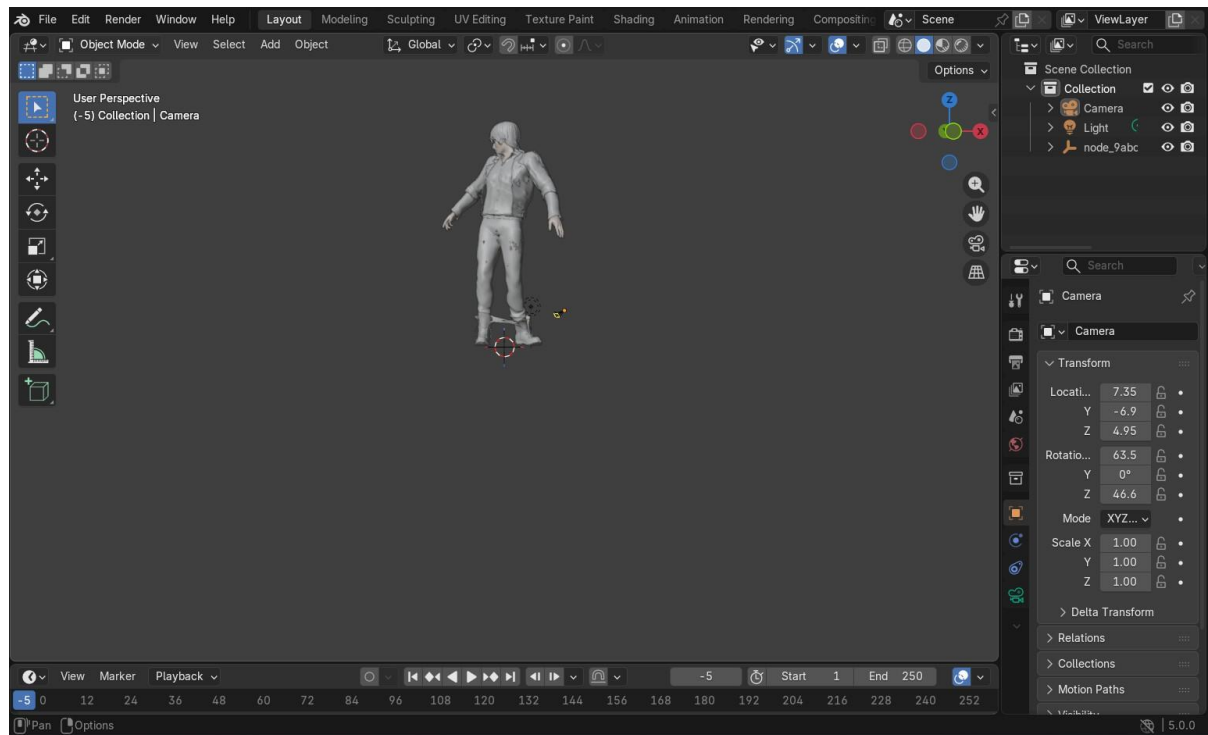


-Utilisation de **Mixamo** pour définir la pose de Timothy Treadwell.









Rendu final :



Puis dans le code nous avons des fonctions pour dessiner à partir de ces modèles 3D :

drawModelWithMaterials(...)

Affiche un modèle OBJ avec ses matériaux et textures.

Pour chaque face, applique la couleur ou la texture du matériau correspondant et utilise les normales pour l'éclairage.

drawAnimatedFox(...)

Affiche un renard animé à partir d'un modèle OBJ.

Le renard effectue un léger mouvement vertical (bobbing) pour simuler la marche, et ses matériaux/textures sont appliqués par face comme dans drawModelWithMaterials.

drawFish(...)

Affiche un poisson en 3D avec texture.

Le poisson est légèrement déplacé verticalement selon une fonction sinusoïdale pour simuler la nage dans l'eau.

drawAnimatedRiver(...)

Affiche une rivière animée à partir d'un modèle OBJ.

Les sommets et coordonnées de texture sont modifiés en continu pour créer un effet de vague et de mouvement de l'eau, tout en appliquant les matériaux du modèle.

- Lumières :

-Initialisation des lumières dans init() :

- GL_LIGHT0 : principale, avec position, couleur ambiante et diffuse.
- GL_LIGHT1 : secondaire, avec position, couleur ambiante, diffuse et spéculaire.

-Activation/désactivation selon les touches (0) ou le type de lumière (4-5-6) dans keyboard().

-Gestion du cycle jour/nuit pour modifier l'éclairage global (dayNightCycle et globalAmbient).

- Animations :

- Fox : mouvement automatique ou manuel, gestion de l'angle des pattes (foxLegAngle) et de la direction (foxX, foxZ, foxAngle).
- Rivière : drawAnimatedRiver() avec waterOffset mis à jour dans timer().
- Poissons : drawFish() animés selon timeElapsed.
- Particules : système de particules avec initParticles() et updateParticles().
- Cycle jour/nuit : animation automatique si autoDayNight est active (dayNightCycle += 0.01f).

- Interactions :

Clavier (keyboard()) :

- Modes d'illumination : 1-2-3.
- Types de lumière : 4-5-6.

- Activer/désactiver shaders : S.
- Pause/play fox : ESPACE.
- Zoom : Z/D.
- Reset caméra : R.
- Cycle jour/nuit : N.
- Activer/ désactiver une lumière : 0

Flèches directionnelles (`specialKeys()`) :

- Déplacement manuel du renard et animation des pattes.

Souris :

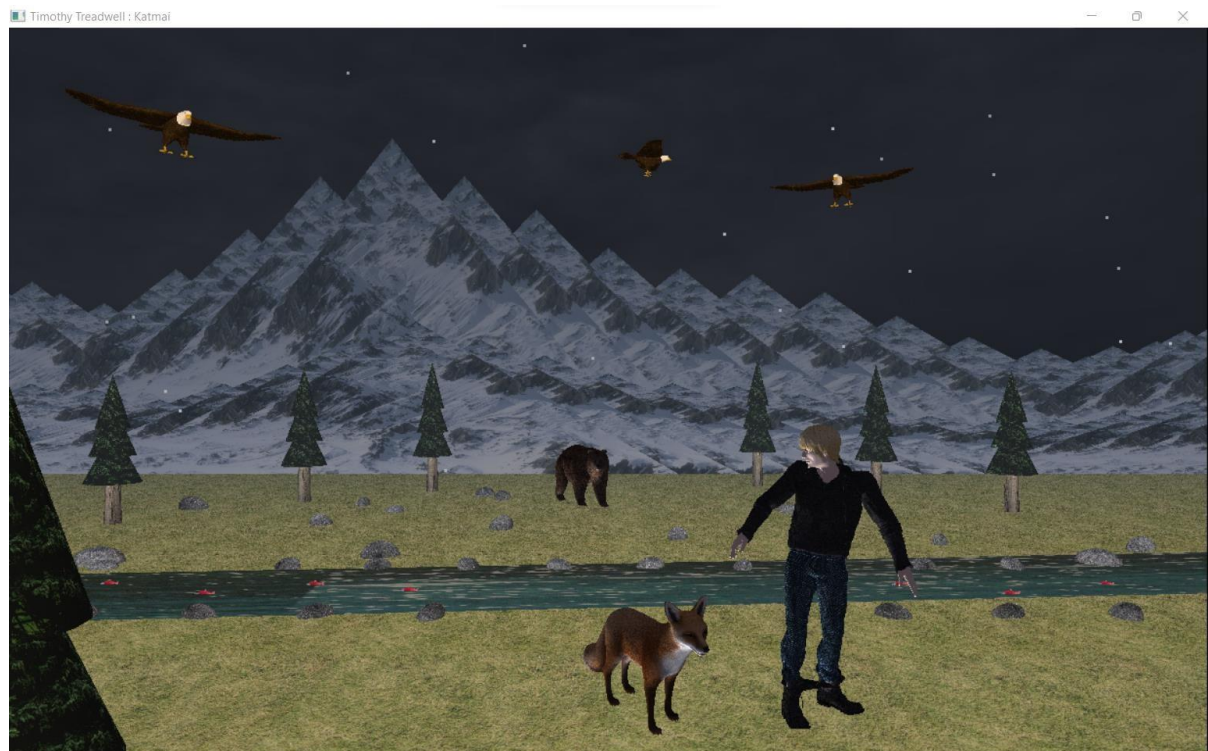
- Rotation caméra avec clic gauche + mouvement (`mouse()` + `mouseMotion()`).
- Zoom avec molette (`mouseWheel()`).

3. Rendu :

Matin :



La nuit tombe...



On allume la lumière, touche 0 :



On essaye plusieurs types de paramètres :

Touche 1-2-3 : Mode d'illumination (1, 2, 3)

Regardons cela à travers le renard :

1 – Lambertien (diffus) :

Rendu mat, lumière diffusée uniformément sur les surfaces.
Pas de reflets brillants.



2 – Spéculaire (miroir) :

Rendu avec reflets nets, surfaces réfléchissantes.



Permet de simuler des surfaces brillantes comme le métal ou l'eau calme.

3 – Glossy (semi-brillant) :
 Entre Lambertien et Spéculaire.
 Surfaces légèrement brillantes, reflets adoucis.



Touche 4-5-6 : Type de lumière (4, 5, 6)

4 – Directionnelle :
 Lumière venant d'une direction infinie, comme le soleil.
 Éclaire toute la scène uniformément.
 Celle par défaut dans la scène.



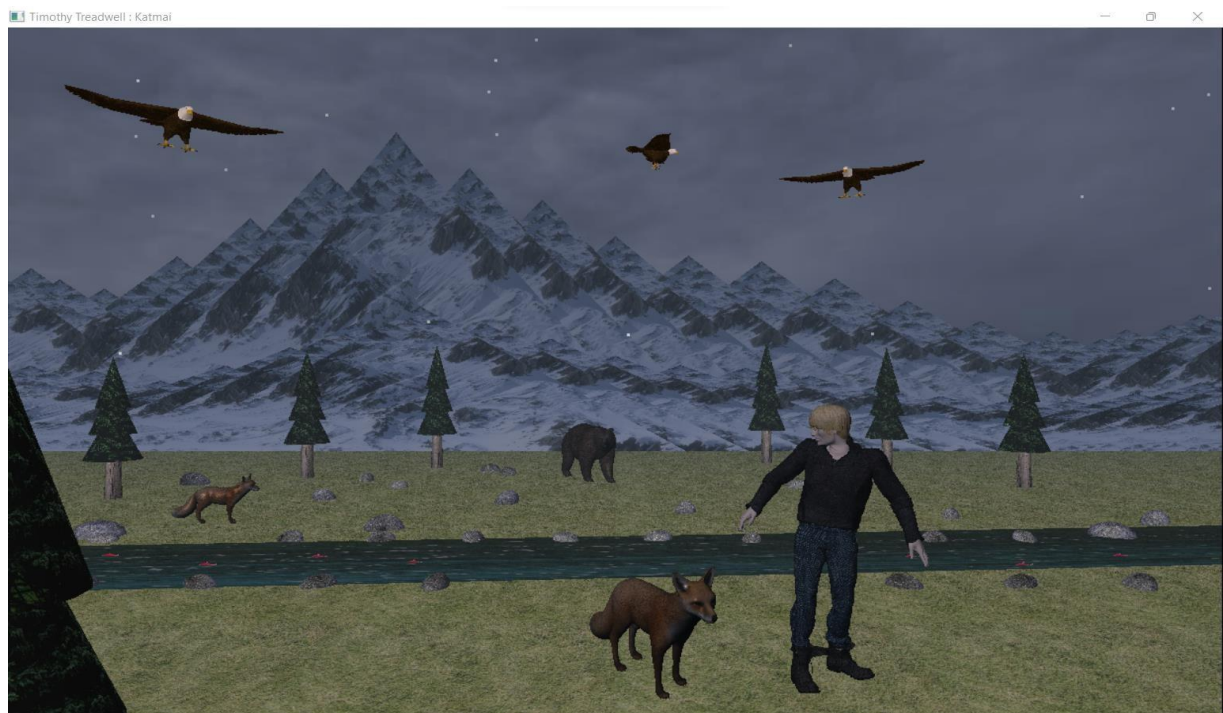
- 5 – Ponctuelle :
Lumière venant d'un point précis, comme une lampe.
L'intensité diminue avec la distance.



- 6 – Spot :
Lumière directionnelle concentrée dans un cône, comme un projecteur.
Possibilité de définir un angle intérieur et extérieur pour le cône.



Rendu sans shaders : cliquer sur la touche S :



Faisons tourner la caméra un petit peu :









Petit détail : ours caché.





Métaphore : L'ours caché représente ce que Timothy ne voyait pas. Il est placé en dehors de son angle de vue, dans une partie de la scène embrumée.

Timothy voyait les ours comme des amis, il était tombé dans l'illusion qu'il avait développé un lien spécial avec ces animaux sauvages.

Sa vision des choses était embrumée. Il ne voyait pas la réalité : un ours affamé voit en l'humain un simple repas.

4. Vérification des contraintes :

Objets (>5) : Ours 1 & 2, Renards, Timothy, Saumon, Aigle, Rivière (objets 3D) , Arbres, Rochers, Sol, Montagnes, Ciel (objets dessinés avec texture)

Réalisme et identification : Écosystème Katmai avec animaux, rivière animée, montagne, arbres et cycle jour/nuit

Textures (>3) : bear.jpg, fox.jpg, grass.jpg, mountain.jpg, sky.jpg, tree.jpg, tronc.jpg, rock.jpg, river.jpg, Mr._Sockeye_baseColor.png

Éclairage (>2 types) :

- GL_LIGHT0 : principale

- GL_LIGHT1 : secondaire activable/désactivable
- Cycle jour/nuit ajustant la luminosité globale
- Possibilité de changer type lumière (directionnelle, ponctuelle, spot)

Mouvement caméra : rotation souris, zoom (Z/D et molette), reset (R)

Interactions (>2) :

- Activer/désactiver lumière secondaire (0)
- Déplacement manuel du renard avec flèches
- Pause/reprise animation renard (ESPACE)
- Changer mode illumination (1-2-3)
- Changer type lumière (4-5-6)
- Touche S pour enlever le rendu shaders